

Extracting 800m PRISM data

August 25, 2025

1 Updates to PRISM

As of March 2025, PRISM has updated their public data database to include 800m resolution for all data variables; however, the 'prism' package has not been updated to include the new resolution. Currently, the only way to extract this data is to use this URL:

```
https://services.nacse.org/prism/data/get/<region>/<res>/<element>/<date><?format=[nc|asc|bil]> ,
```

where

- **<region>** is the region you want to extract data to (in our case, 'us')
- **<res>** is the resolution you want to extract to (800m)
- **<element>** is the climate variable (e.g., 'tmean'; see their website for the full list)
- **<date>** is the date you want to extract for, in the format **YYYYMMDD** for daily data, **YYYYMM** for monthly data, or **YYYY** for annual data
- **?format=[nc|asc|bil]** is the format you want the data in; for rasters we want to use **bil**

You'll notice that you can't specify a date *range* in this URL. Because of this, we we will need to build a web scraper to download a range of dates from their server. To do this, we need a couple things:

- a text editor to write the scraper (e.g., VSCode)
- strawberry perl (<https://strawberryperl.com/>) to execute the scraper
- software of choice to extract the climate files in bulk (7Zip, WinRAR, etc.)

2 Building the scraper

Conveniently, the team over at PRISM provided a template for us to use [1]. We'll need to modify it a bit to get it to work for our purposes, but the skeleton is there.

Create a new file in your text editor and paste the following code:

```
#!/usr/bin/perl
use strict;
use warnings;
use DateTime;
my $day;
my $clim_var = 'ppt';
my $base_url = 'https://services.nacse.org/prism/data/get/us/4km';
my $start = DateTime->new( day => 1, month => 1, year => 1999 );
my $stop = DateTime->new( day => 31, month => 12, year => 1999 );
while($start <= $stop) {
    $day = $start->strftime('%Y%m%d'); #place date in proper format
    system("wget --content-disposition $base_url/$clim_var/$day");
    sleep 2; #to be nice to our server
    $start->add(days => 1);
}
```

By default, the skeleton provided will extract data in the netCDF format at a 4km resolution in 1999, so we need to (1) modify the end of the `my $base_url` call, (2) add a line that specifies

the format we want, e.g. my \$format = '?format=bil' and (3) specify a relevant date range in my \$start and my \$stop (August 5, 2024 to September 13, 2024, for example).

```
#!/usr/bin/perl

use strict;

use warnings;

use DateTime;

my $day;

my $clim_var = 'ppt';

my $base_url = 'https://services.nacse.org/prism/data/get/us/800m';

my $start = DateTime->new( day => 5, month => 8, year => 2024 );

my $stop = DateTime->new( day => 13, month => 9, year => 2024 );

my $format = '?format=bil'; # have to specify that we want bil so we can raster stack in R

while($start <= $stop) {

    $day = $start->strftime('%Y%m%d'); #place date in proper format

    system("wget --content-disposition $base_url/$clim_var/$day");

    sleep 2;

    $start->add(days => 1);

}
```

We'll be using curl, which is built into the Windows 10+ command line, to execute this script. In doing this, though, we need to modify the system call to use curl instead of wget, as follows:

```
#!/usr/bin/perl

use strict;

use warnings;

use DateTime;

my $day;

my $clim_var = 'ppt';
```

```

my $base_url = 'https://services.nacse.org/prism/data/get/us/800m';
my $start = DateTime->new( day => 5, month => 8, year => 2024 );
my $stop = DateTime->new( day => 13, month => 9, year => 2024 );
my $format = '?format=bil'; # have to specify that we want bil so we can raster stack in R
while($start <= $stop) {
    $day = $start->strftime('%Y%m%d'); #place date in proper format
    system("curl -L -OJ \"$base_url/$clim_var/$day$format\""); # use curl in system call
    # -L follows redirects, -O saves the file with the original name, -J uses the
    # Content-Disposition header to save the file
    sleep 2;
    $start->add(days => 1);
}

```

Save this .pl file in the same directory that you want to save the climate data in. From here you can either create multiple versions for each climate variable (e.g. `PRISMScraperTmax.pl`, `PRISMScraperTmean.pl`, etc.), or keep a single .pl and modify the `my $clim_var` variable in the script after each system call to extract different variables.

2.1 Running the scraper in command line

First, open up the command line by pressing Win + R, typing `cmd`, and hitting enter. From there, change the directory to the one where you saved the .pl file by using the `cd` command, followed by the path.

For example, if you saved the file in `C:\Users\fake path\ClimateDataPath`, your command would look like this:

```
cd C:\Users\fake path\ClimateDataPath
```

You should see the path change to the one you've specified. Next, you need to run the script using Perl. If you installed Strawberry Perl, you should be able to run the script by typing:

```
perl PRISMScraperTmax.pl
```

You should see the command line start to download files, one for each day in the date range you specified. As mentioned before, the files will be saved to the same directory as the .pl file, so make sure you have enough space on your hard drive to store them all. Extract all files into a single, main folder (e.g., `C:/Users/fake path/ClimateDataPath/MainFolder`), using a software that is capable of bulk extracts like 7Zip, as this will make it easier to stack them in R later.

3 Stacking and data extraction in R

Create a list of the files you just downloaded by using the `list.files()` function and stack them as follows:

```
library(raster)

# climate data directory from section 2.2
dir <- "C:/Users/fake path/ClimateDataPath/MainFolder"

# lists all .bil files in the directory
bils <- paste(dir,"/",list.files(pattern = "\\\\.bil$"), sep = "")

climate_data <- stack(
  bils
)
```

From here we can extract the data to the points of interest.

```
# extract project coordinates from raster stack
climate_data@crs@projargs <- "+proj=longlat +ellps=WGS84 +datum=WGS84"
climate_crs <- climate_data@crs@projargs

# load OR coordinates and project to same CRS as climate data
```

```

coordinates(or.df) <- c("Long", "Lat")
proj4string(or.df) <- CRS("+proj=longlat +ellps=WGS84 +datum=WGS84")

climate_or <- data.frame(
  # column of coordinates
  coordinates(or.df),
  # column of site names
  or.df$Site,
  # EXAMPLE: extracted data to the point
  extract(climate_data, or.df)
)

### cleaning up columns
climate_or <- climate_or %>%
  gather(date, value, 4:ncol(climate_or))

# the column header includes the date and data type, but also some other metadata that we don't need
# remove extra info from the column header
climate_or$date <- gsub('prism_', '', climate_or$date) %>%
  gsub('us_30s_', '', .)

# split header into type (ppt, tmean, tmax, etc.) and date
climate_or <- separate(climate_or, 'date',
  into = c('type', 'date'),
  sep = '_')
climate_or$date <- ymd(as.numeric(climate_or$date))
climate_or$date <- as.Date(climate_or$date, format = "%Y/%m/%d")

```

```
# reshape data-- make a separate column for temperature and precipitation
climate_or <- unique(climate_or)
climate_or <- climate_or %>%
  spread(type, value) %>%
  dplyr::rename(Site = or.df.Site)

## save
write_rds(climate_or, "Talent with climate.rds")
```

You now have a data frame with daily climate variables extracted to the point ready to be joined with your project data (i.e., the network).

References

- [1] *Gridded Dataset Downloads via Web Service 2025*. 2025.